

# FundsRoom ERP — CRM & Operations Portal

A full-stack ERP platform for customer management, inventory operations, stock tracking, and sales challan processing.

## 1. PROJECT OVERVIEW

FundsRoom ERP is a full-stack enterprise operations portal designed to manage customers, products, inventory, stock movements, and sales challans through a role-based workflow.

### Main Objectives:

- Centralize customer information
- Manage products and inventory
- Track stock movements
- Create and process sales challans
- Enforce role-based access
- Maintain reliable inventory updates

## 2. KEY FEATURES

### Authentication & Authorization

- JWT authentication
- bcrypt password hashing
- role-based authorization
- protected API routes
- role-specific access
- open user registration with role selection

### Customer CRM

- customer creation
- customer updates
- customer details
- search
- filtering
- pagination
- follow-up dates
- notes

### Product & Inventory

- product creation
- SKU management
- product updates
- inventory tracking
- low-stock detection
- stock IN/OUT movements
- stock movement history

- prevention of negative stock
- prevention of direct stock modification

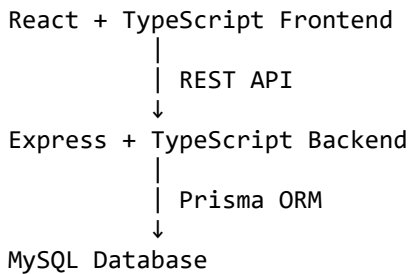
## Sales Challans

- draft challans
- challan items
- product snapshots
- challan confirmation
- challan cancellation
- automatic stock deduction
- stock movement audit
- atomic confirmation transaction

## 3. TECHNOLOGY STACK

| Category     | Technologies                                                             |
|--------------|--------------------------------------------------------------------------|
| Frontend     | React 19, TypeScript, Vite, Tailwind CSS, React Router DOM, Lucide React |
| Backend      | Node.js, Express.js, TypeScript, Prisma, Zod, JWT, bcrypt                |
| Database     | MySQL                                                                    |
| Architecture | REST API, Client-Server architecture                                     |

## 4. SYSTEM ARCHITECTURE



### Frontend handles:

- UI
- routing
- authentication state
- API communication
- role-based UI visibility

### Backend handles:

- authentication
- authorization
- validation
- business logic
- database operations
- transactions

### Database handles:

- users
- customers
- products
- stock movements
- challans
- challan items

## 5. DATABASE DESIGN

### User

**Purpose:** Authentication and system users.

- **Fields:** id, name, email, password, role, createdAt, updatedAt
- **Roles:** ADMIN, SALES, WAREHOUSE, ACCOUNTS

### Customer

**Purpose:** Manages client profiles and business information.

- **Important Fields:** id, name, mobile, email, businessName, gstNumber, customerType, address, status, followUpDate, notes, createdAt, updatedAt

### Product

**Purpose:** Represents inventory items with stock levels.

- **Fields:** id, name, sku, category, unitPrice, currentStock, minimumStock, warehouse, createdAt, updatedAt

### StockMovement

**Purpose:** Logs IN and OUT transactions for products.

- **Fields:** id, productId, quantity, movementType (IN/OUT), reason, createdBy, createdAt

### Challan

**Purpose:** Delivery note or invoice header linking customers and items.

- **Fields:** id, challanNumber, customerId, status (DRAFT, CONFIRMED, CANCELLED), totalQuantity, createdBy, createdAt, updatedAt

### ChallanItem

**Purpose:** Line items of a challan capturing a snapshot of product state and quantities.

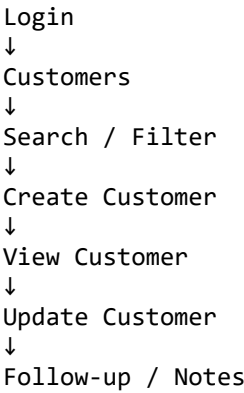
- **Fields:** id, challanId, productId, productName, sku, unitPrice, quantity, total

## 6. ROLE-BASED ACCESS

| Entity / Route  | ADMIN                                     | SALES     | WAREHOUSE             | ACCOUNTS |
|-----------------|-------------------------------------------|-----------|-----------------------|----------|
| Customers       | Full Access Read, Create, Update          |           | No Access             | Read     |
| Products        | Full Access Read                          |           | Read                  | Read     |
| Stock Movements | Full Access                               | No Access | Read, Create (IN/OUT) |          |
| Challans        | Full Access Read, Create, Confirm, Cancel |           |                       | Read     |

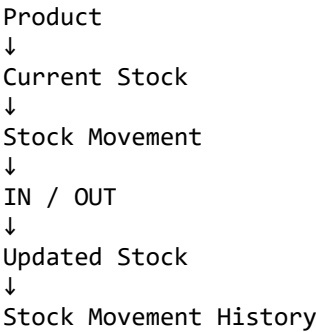
Permissions are strictly enforced via backend middleware on every relevant route.

## 7. CUSTOMER CRM WORKFLOW



- **Validation:** Utilizes Zod for strict payload validation before hitting the database.
- **Format checks:** Enforces Indian mobile number validation format.
- **Pagination:** Endpoints are optimized with server-side pagination to handle large datasets.

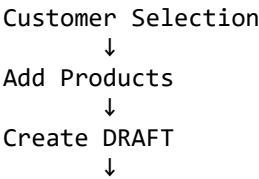
## 8. INVENTORY WORKFLOW



**Important Implementation Details:**

- Direct currentStock modification through the general product update flow is restricted.
- All stock changes must occur exclusively through traceable stock movement operations.
- Backend logic ensures that OUT operations cannot reduce the stock below zero.

## 9. SALES CHALLAN WORKFLOW



Review



CONFIRM



Stock Deduction



OUT Stock Movement



CONFIRMED

- **DRAFT:** Does not affect inventory.
- **CONFIRMED:** Deducts inventory.
- **CANCELLED:** Does not deduct inventory.

## 10. TRANSACTION SAFETY

When a challan is confirmed, the backend performs the following sequence atomically:

1. Backend starts a Prisma transaction.
2. Each product's available stock is checked.
3. If sufficient stock exists, stock is decremented.
4. Corresponding stock movement records are created.
5. Challan status is updated to **CONFIRMED**.
6. If any item fails because of insufficient stock, the transaction is rolled back completely.

**Result:** Either the complete confirmation succeeds, or the inventory remains unchanged. Atomic conditional stock updates are used to prevent negative stock during concurrent operations.

## 11. API DOCUMENTATION

### Authentication:

- **POST /api/auth/login:** Authenticate and receive JWT.
- **GET /api/auth/me:** Retrieve current user profile (requires Auth).

### Customers:

- **GET /api/customers:** List customers (ADMIN, SALES, ACCOUNTS).
- **POST /api/customers:** Create customer (ADMIN, SALES).
- **GET /api/customers/:id:** Get customer details (ADMIN, SALES, ACCOUNTS).
- **PUT /api/customers/:id:** Update customer (ADMIN, SALES).

### Products:

- **GET /api/products:** List products (ALL).
- **POST /api/products:** Create product (ADMIN).
- **GET /api/products/:id:** Get product details (ALL).
- **PUT /api/products/:id:** Update product (ADMIN).
- **POST /api/products/:id/stock-movements:** Add IN/OUT movement (ADMIN, WAREHOUSE).
- **GET /api/products/:id/stock-movements:** View movement history (ADMIN, WAREHOUSE, ACCOUNTS).

### Challans:

- **GET /api/challans:** List challans (ALL).

- POST /api/challans: Create draft challan (ADMIN, SALES).
- GET /api/challans/:id: Get challan details (ALL).
- POST /api/challans/:id/confirm: Confirm challan and deduct stock (ADMIN, SALES).
- POST /api/challans/:id/cancel: Cancel draft challan (ADMIN, SALES).

## 12. VALIDATION & ERROR HANDLING

- **Zod Validation:** All incoming request bodies are validated against strict Zod schemas.
- **Centralized Error Handling:** Catch-all error middleware normalizes error responses.
- **Authentication Errors:** 401 Unauthorized for missing/invalid tokens.
- **Authorization Errors:** 403 Forbidden for restricted actions.
- **Not-Found Responses:** 404 Not Found for missing IDs.
- **Duplicate Handling:** Graceful rejection of duplicate SKUs.
- **Insufficient Stock:** Rejection with a clear message if stock deduction fails.
- **Invalid Input:** Detailed field-level errors returned by Zod.

## 13. FRONTEND STRUCTURE

### Pages:

- LoginPage
- DashboardPage
- CustomersPage
- CustomerDetailPage
- ProductsPage
- ProductDetailPage
- ChallansPage
- CreateChallanPage
- ChallanDetailPage

The frontend utilizes reusable UI components and React Context (e.g. AuthContext) for global state. Role-based visibility is implemented so users only see navigation elements they have access to.

## 14. BACKEND STRUCTURE

```
server/src/
├── controllers/  # Request handlers and business logic
├── middleware/   # Auth, role enforcement, and error handling
├── routes/       # Express router definitions
├── validators/   # Zod schema definitions
├── utils/        # Helper functions (e.g. JWT signing)
└── server.ts     # Application entry point
```

```
server/prisma/
├── schema.prisma # Database models and relations
└── seed.ts       # Initial database seeding script
```

## 15. PROJECT STRUCTURE

```
FundFlow-ERP-CRM-Operations-Portal/
├── client/
│   ├── src/
│   └── public/
```

```

├── package.json
├── server/
│   ├── src/
│   ├── prisma/
│   └── package.json
├── README.md
├── PROJECT_DOCUMENTATION.md
└── .gitignore

```

## 16. LOCAL SETUP

### Backend

```

cd server
npm install

```

Configure `.env` using `.env.example` (MySQL must be running locally):

```

npx prisma generate
npx prisma migrate dev
npx prisma db seed
npm run dev

```

### Frontend

```

cd client
npm install
npm run dev

```

## 17. ENVIRONMENT VARIABLES

### Backend Required Variables (Example only):

- `PORT=5000`
- `DATABASE_URL="mysql://USER:PASSWORD@localhost:3306/DATABASE"`
- `JWT_SECRET="your_jwt_secret_key"`
- `CLIENT_URL="http://localhost:5173"`

## 18. DEMONSTRATION WORKFLOW

### Recommended Demonstration

For case study evaluation, we recommend this linear flow:

1. Login
2. Dashboard
3. Customer creation
4. Product management
5. Stock movement
6. Create challan
7. Save challan as draft
8. Confirm challan
9. Verify stock deduction

10. Verify stock movement history
11. Demonstrate role-based access

## 19. DESIGN & UX

The frontend is built with Tailwind CSS and Lucide React, providing a clean and professional dashboard interface. Data is presented in structured tables with state indicators like colored badges for Challan status and low stock warnings. The UX is optimized for readability and fast interactions.

## 20. POSTMAN COLLECTION & API DOCS

A complete Postman collection is included in the repository at `postman/FundsRoom-ERP.postman_collection.json`. It provides fully configured endpoints for authentication, CRM, inventory, and challans. It utilizes the `{{baseUrl}}` variable and dynamic `{{token}}` injection after login for seamless evaluation.

## 21. KNOWN LIMITATIONS

- **File Uploads:** Attachments and product images are intentionally omitted.
- **Reporting:** Advanced analytics and PDF generation are placeholders.
- **Notifications:** Email notifications on registration or challan status changes are not implemented.
- **Deployment:** The provided setup is strictly for local case-study evaluation.

## 22. CONCLUSION

FundsRoom ERP provides a centralized workflow for CRM, inventory, stock movement, and sales challan management. With robust authentication, fine-grained authorization, strict payload validation, and transaction-safe inventory operations, it serves as a reliable enterprise portal for core business operations.